Semantic Debt

A Two-Axis Ontology of Meaning Failure in Software, Governance, and Agent-Built Systems

The system may still speak, but can its meanings still be recovered?

Author
Adam Ableman
Constitutional Computing
Contact: ableman.research@gmail.com

Abstract

Software systems do not only accumulate technical debt. They accumulate semantic debt: the future cost of meaning that is underdefined, misplaced, contradictory, stale, fragmented, opaque, overcoupled, overclaimed, inadmissible, or only recoverable through inference. Technical debt concerns the burden created by implementation shortcuts, architectural compromises, postponed refactoring, or quality degradation. Semantic debt concerns a different layer: the decay of recoverable meaning. A system may compile, pass tests, and remain operational while its terms drift, its categories collapse, its authority assumptions hide, its decisions vaporize, its evidence becomes inadmissible, and its claims outrun what its artifacts can support.

This paper proposes a two-axis ontology of semantic debt. The first axis defines seven semantic subdomains: lexical debt, ontological debt, boundary debt, procedural debt, evidentiary debt, authority debt, and claim debt. The second axis defines nine debt modes: underdefinition, contradiction, fragmentation, staleness, opacity, overcoupling, inadmissibility, overclaim, and lossy translation. Together, these axes give semantic debt a coordinate system. A debt item is not merely named. It is located by what kind of meaning is failing and by how that failure manifests.

The paper gives special attention to semantic fragmentation. Fragmentation is not a peer subdomain beside lexical, ontological, boundary, procedural, evidentiary, authority, and claim debt. It is a cross-cutting distribution failure that can affect any of them. The precise motivating subtype is semantic locality debt: the cost incurred when meaning exists somewhere in a system but not in a stable, canonical, locally recoverable home.

The ontology is intended to be operational. The semantic debt matrix leads directly to governing artifacts: glossaries, non-claims files, claim ledgers, standing graphs, migration contracts, receipt schemas, and semantic debt ledgers. These artifacts turn semantic debt from an interpretive complaint into a governable

engineering object. The framework is especially relevant to agent-built systems, where plausible automation can introduce acronym drift, status collapse, authority inference, claim inflation, and boundary leakage while preserving apparent functionality.

1. Introduction: The Hidden Debt of Meaning

A software system can continue functioning while its meaning decays.

The tests still pass. The package still builds. The API still responds. The dashboard is still green. Yet the system may no longer be able to answer basic semantic questions:

- What does this status mean?
- Is this artifact validated, admitted, executed, or standing-bearing?
- What authority permitted this effect?
- What claim does this receipt support?
- What is explicitly not claimed?
- Which policy version governed this transition?
- Was this refusal an error, or successful boundary preservation?
- Did this refactor preserve meaning, or only behavior?
- Can this system be migrated without semantic loss?

These are questions of semantic recoverability. When a system cannot answer them from stable artifacts, it has accumulated semantic debt.

Technical debt has become a central metaphor in software engineering. Kruchten, Nord, and Ozkaya describe technical debt as a metaphor that moved beyond rhetoric into a practical framework for understanding tradeoffs, future change, and explicit management of development compromises. Modern technical debt research likewise emphasizes that unmanaged debt threatens long-lived and complex systems and requires explicit management.

Semantic debt is adjacent to this tradition but points to a different layer. It is not primarily the cost of bad code. It is the cost of lost, unstable, scattered, or inadmissible meaning.

A useful contemporary definition of semantic debt describes it as the accumulated cost created when terms, categories, entities, examples, translations, and relationships remain underdefined, contradictory, outdated, or weakly maintained, forcing readers and AI systems to infer structure that should have been declared. This paper generalizes that insight from interpretive corpora into software systems, governance artifacts, agentic development, and constitutional computing.

The central claim is simple:

> A system may still run after its meaning has become expensive to recover.

2. Hard Definitions

This paper uses several terms strictly. These definitions are introduced once here and used consistently afterward.

2.1 Semantic Debt

Semantic debt is the accumulated future cost of meaning that is underdefined, misplaced, contradictory, stale, fragmented, opaque, overcoupled, overclaimed, inadmissible, or only recoverable through inference.

A system has semantic debt when its semantic structure cannot be reliably recovered from its artifacts.

Semantic debt is not technical debt. Technical debt concerns implementation compromises. Semantic debt concerns meaning failure.

Semantic debt is not documentation debt. Documentation debt concerns missing or stale explanatory material. Semantic debt concerns the system's meanings themselves, whether those meanings are expressed in code, schema, tests, receipts, policies, names, claims, decisions, runtime traces, or documentation.

Semantic debt is not ambiguity. Ambiguity is one possible mode of semantic debt. Semantic debt also includes category collapse, authority concealment, evidence gaps, claim inflation, policy drift, stale standing, and lossy migration.

2.2 Semantic Fragmentation

Semantic fragmentation is a cross-cutting debt mode in which meaning is scattered across multiple surfaces without a stable canonical home.

Fragmentation can affect terms, categories, boundaries, procedures, evidence, authority, and claims. It is not one subdomain beside the others. It is a distribution failure that may appear inside any subdomain.

2.3 Semantic Locality Debt

Semantic locality debt is the cost incurred when meaning exists somewhere in a system but not in a stable, canonical, locally recoverable home.

This is the precise motivating subtype behind the original concern about semantic fragmentation. The problem is not merely that meaning is distributed. The problem is that the system lacks an authoritative place where a future human, tool, or agent can recover that meaning without archaeology.

The governing principle is:

> Every meaningful distinction should have a stable home.

2.4 Semantic Recoverability

Semantic recoverability is the degree to which a system's meanings can be reconstructed from stable artifacts without relying on unstated human memory, guesswork, or convention.

A system has high semantic recoverability when a future reviewer can identify its terms, categories, boundaries, procedures, evidence, authority, claims, non-claims, and standing from canonical artifacts.

A system has low semantic recoverability when those answers require scattered code reading, stale documents, runtime trace interpretation, historical chats, and developer memory.

2.5 Admissibility

Admissibility is the property by which a semantic artifact can support a claim under explicit rules.

An artifact is not admissible merely because it exists. A log, trace, test result, or screenshot becomes evidence only when it is bound into a receipt, manifest, attestation, or other canonical artifact capable of supporting or refusing a claim.

In short:

```
non-binding trace != evidence
artifact existence != admissibility
receipt existence != standing
```

2.6 Standing

Standing is a governed status granted to a package, artifact, receipt, transition, claim, or subsystem under explicit rules.

Standing is not confidence. Standing is not test success. Standing is not execution. Standing is a scoped status that persists or changes according to standing transition rules.

A major source of semantic debt is the collapse of validation, admission, execution, and standing into one vague notion of "works."

3. The Two-Axis Ontology

The core contribution of this paper is a two-axis ontology.

Semantic debt is classified along two axes:

1. Semantic subdomain: what layer of meaning is indebted?
2. Debt mode: how is that layer indebted?

The first axis identifies the kind of semantic material involved. The second identifies the failure pattern.

```
semantic debt item
  = subdomain + mode + affected artifact + consequence + repayment
  action
```

The seven subdomains are:

```
lexical
ontological
boundary
procedural
evidentiary
authority
claim
```

The nine modes are:

```
underdefinition
contradiction
fragmentation
staleness
opacity
overcoupling
inadmissibility
overclaim
lossy translation
```

Together they form a seven by nine diagnostic coordinate system. A single debt instance may occupy several cells, but the axes remain distinct.

Examples:

```
Unexplained acronym GESD
  = lexical debt + underdefinition

Package admission treated as execution standing
  = ontological debt + overclaim

Policy check hidden inside filesystem adapter
  = authority debt + opacity
  = boundary debt + overcoupling

Receipt exists but does not bind prior state
  = evidentiary debt + inadmissibility

Python-to-C migration loses refusal modes
  = boundary debt + lossy translation
  = procedural debt + inadmissibility
```

The matrix is the step from concept to discipline. It lets developers, reviewers, tools, and agents locate debt precisely enough to govern it.

4. Axis One: Seven Semantic Subdomains

The seven subdomains are distinct in presentation even though real debt instances often occupy more than one.

4.1 Lexical Debt

Lexical debt is debt in the naming layer: terms, acronyms, labels, identifiers, field names, and code names.

Examples:

- one term has multiple meanings;
- multiple terms refer to the same concept without distinction;
- acronyms proliferate without a glossary;
- stale field names preserve old standing;
- deprecated names remain active without migration mapping;
- "evidence," "log," "trace," and "receipt" are used interchangeably.

Primary remedies:

- glossary;
- acronym registry;
- deprecated-name table;
- field-definition catalog;
- canonical term IDs;
- glossary coverage checks.

Lexical debt is the first fog. If terms cannot be trusted, stronger structures built on them inherit mist.

4.2 Ontological Debt

Ontological debt is debt in the category layer: types, entity boundaries, status distinctions, and kinds of things.

Examples:

- validation is treated as admission;
- admission is treated as execution;
- execution is treated as standing;
- standing is treated as maturity;
- receipt existence is treated as evidence sufficiency;
- fixture-scoped status is treated as broad status;
- package status, execution status, and claim status are collapsed into a generic `status`.

The critical distinction is:

```
validation != admission
admission != execution
```

```
execution != standing
standing != broad maturity claim
```

Primary remedies:

- explicit type model;
- standing enum;
- status-transition graph;
- "this is not that" tables;
- schema-level category constraints;
- claim-to-standing mapping;
- migration records for category changes.

Ontological debt is the debt of mistaken kinds.

4.3 Boundary Debt

Boundary debt is debt in the jurisdiction layer: contexts, packages, adapters, domains, schemas, and substrates.

Examples:

- a UI status label implies governance standing;
- a test fixture result implies package admission;
- a Python package claim leaks into C-D.MS receipt standing;
- local validation becomes global claim;
- a domain term crosses bounded contexts without translation;
- a port or adapter boundary is treated as a convenience seam rather than an authority boundary.

Primary remedies:

- bounded contexts;
- ports and adapters;
- jurisdiction labels;
- context maps;
- translation records;
- substrate migration contracts;
- scope fields;
- non-claims.

Boundary debt is the debt of unmarked crossings.

4.4 Procedural Debt

Procedural debt is debt in the transition layer: state changes, decisions, migrations, repairs, admissions, refusals, and standing changes.

Examples:

- a package becomes "admitted" without an admission decision record;

- a roadmap advances without a standing transition record;
- a repair changes evidence but does not distinguish repair standing from new standing;
- a migration changes data but not the semantic status of migrated fields;
- a README changes status while the canonical index remains stale;
- a policy refactor changes behavior without a migration record.

Primary remedies:

- decision records;
- admission, refusal, and defer records;
- standing transition records;
- migration receipts;
- repair receipts;
- roadmap-to-index consistency checks.

Procedural debt is the debt of unrecorded passage.

4.5 Evidentiary Debt

Evidentiary debt is debt in the proof layer: receipts, manifests, traces, provenance, indexes, hashes, and auditability.

Examples:

- logs exist but are not receipt-bound;
- test output is not tied to artifact hashes;
- a build result lacks source, environment, and policy binding;
- a receipt omits prior state or next state;
- a verification report is hash-stable but semantically stale;
- evidence artifacts exist but are not indexed;
- a claim references "the run" but no canonical run receipt exists.

This paper reserves the word evidence for receipt-bound artifacts. Runtime logs, console output, screenshots, metrics, and ordinary traces are non-binding traces until bound into canonical evidence.

Primary remedies:

- receipts;
- manifests;
- hash seals;
- content addressing;
- provenance graphs;
- evidence chains;
- fixture replay;
- claim-to-evidence maps;
- admissibility predicates.

Evidentiary debt is the debt of behavior without proof.

4.6 Authority Debt

Authority debt is debt in the permission layer: policy, capability, warrant, execution authority, and claim authority.

Examples:

- a function writes to disk because it can;
- a subprocess launches because the script reached that line;
- a validator mutates state;
- a package executes because it passed validation;
- a tool infers permission from availability;
- a policy decision is hidden inside an adapter;
- a compiler is invoked without execution authority;
- a claim is made because an artifact exists, not because standing was granted.

Primary remedies:

- explicit governed context;
- authority references;
- capability sets;
- policy surfaces;
- adapter mediation;
- least-authority execution;
- refusal codes;
- admission records.

Authority debt is the debt of action without warrant.

4.7 Claim Debt

Claim debt is debt in the assertion layer: claims, non-claims, standing claims, maturity claims, equivalence claims, and scope.

Examples:

- "validated" is presented as "admitted";
- "fixture-scoped" is presented as general;
- "receipt exists" is presented as "schema-conformant receipt standing";
- "runs locally" is presented as governed execution;
- "migration succeeded" is presented as equivalence;
- "one self-reference passed" is presented as self-hosting;
- a README implies a stronger status than the canonical index supports.

Primary remedies:

- non-claims file;
- claim ledger;
- claim-to-evidence mapping;
- standing transition records;

- scope tags;
- active blocking claims;
- explicit revocation of non-claims.

Claim debt is the debt of saying more than the system can support.

5. Axis Two: Nine Debt Modes

The seven subdomains identify what layer is indebted. The nine modes identify how the debt manifests.

5.1 Underdefinition

Underdefinition occurs when a semantic object exists but lacks sufficient definition.

Examples:

- acronym without expansion;
- status without transition rules;
- refusal code without meaning;
- policy version without policy surface;
- receipt field without binding semantics.

Typical remedies:

- definitions;
- schemas;
- glossary entries;
- transition rules;
- field constraints;
- examples and counterexamples.

5.2 Contradiction

Contradiction occurs when two or more semantic surfaces assert incompatible meanings.

Examples:

- README says “preflight only” while a decision record grants standing;
- batch index says the next rung is the current rung;
- one schema treats `status` as validation while another treats it as standing;
- non-claims block a claim that another artifact asserts as granted;
- policy permits an effect that the authority model forbids.

Typical remedies:

- canonical source hierarchy;
- consistency checks;
- contradiction detector;

- claim ledger;
- standing graph validation;
- stale artifact retirement.

5.3 Fragmentation

Fragmentation occurs when meaning is scattered across multiple surfaces without a stable canonical home.

Examples:

- authority rules split across code, README, tests, and operator memory;
- proof split across logs, reports, screenshots, and filenames;
- acronym meaning distributed across transcripts;
- standing status spread across roadmap, index, seal, and README;
- refusal semantics appear only in negative tests and comments.

Typical remedies:

- stable canonical homes;
- glossary entries;
- claim ledger;
- standing graph;
- receipt schema;
- non-claims file;
- artifact index;
- semantic debt ledger.

Fragmentation is cross-cutting. Semantic locality debt is the specific form of fragmentation where the missing element is a stable canonical home.

5.4 Staleness

Staleness occurs when a semantic surface was once correct but no longer matches current standing, behavior, policy, schema, or evidence.

Examples:

- README says the next milestone is a batch already completed;
- an index field preserves an old status name;
- a glossary definition survives after the term changes;
- a non-claim remains active after an admissible revocation but was not updated;
- a policy description references an obsolete capability model.

Typical remedies:

- freshness checks;
- canonical index comparison;
- standing timestamp validation;
- stale term detection;

- deprecation records;
- update obligations in release gates.

5.5 Opacity

Opacity occurs when meaning is present but hidden inside implementation details, conventions, or side effects.

Examples:

- policy decision embedded in an adapter if statement;
- authority inferred from an environment variable;
- migration meaning encoded only in SQL;
- refusal code constructed dynamically with no registry;
- claim implied by a script name.

Typical remedies:

- extracted policy surfaces;
- explicit registries;
- typed models;
- static analysis;
- receipt emission;
- code-to-artifact mapping;
- decision records.

5.6 Overcoupling

Overcoupling occurs when meanings that should remain distinct are bound together so tightly that one cannot change without silently affecting the other.

Examples:

- policy and execution share one function;
- state migration and standing migration are coupled;
- logging and evidence emission use the same path;
- validation and admission use the same status field;
- UI labels drive governance decisions;
- adapter availability implies authority.

Typical remedies:

- separation of core and adapters;
- typed status distinctions;
- policy interfaces;
- effect requests;
- standing transition records;
- claim layers;
- bounded contexts.

5.7 Inadmissibility

Inadmissibility occurs when a semantic artifact cannot support the claim it is asked to support.

Examples:

- receipt lacks state hashes;
- test report lacks artifact hashes;
- policy decision lacks authority reference;
- migration record lacks source and target hashes;
- claim references non-binding traces;
- standing transition lacks transition record;
- refusal has no typed code.

Typical remedies:

- admissibility predicates;
- schema checks;
- receipt binding;
- artifact hashes;
- authority references;
- claim-to-evidence validation.

5.8 Overclaim

Overclaim occurs when a claim exceeds its evidence, standing, scope, or authority.

Examples:

- local fixture success becomes broad reliability;
- package admission becomes execution standing;
- execution standing becomes execution occurred;
- schema validation becomes schema-conformant standing;
- migration success becomes equivalence;
- equivalence becomes self-hosting.

Typical remedies:

- non-claims;
- claim ledger;
- scope fields;
- standing graph;
- refusal of unsupported claims;
- maturity-level separation.

5.9 Lossy Translation

Lossy translation occurs when software, data, schemas, policies, or receipts move across representations while losing semantic obligations.

Examples:

- Python behavior is ported to C but refusal semantics disappear;
- JSON fields migrate to a database without preserving constraints;
- receipt schema changes but authority binding becomes optional;
- domain model becomes API contract but loses bounded-context assumptions;
- governance policy becomes prose and loses totality;
- natural-language claim becomes machine status without scope.

Typical remedies:

- migration contracts;
- mapping functions;
- dual hashes;
- equivalence obligations;
- policy refinement checks;
- receipt preservation checks;
- loss reports.

6. The Semantic Debt Matrix

The semantic debt matrix is the center of the ontology. It combines seven subdomains with nine modes. It is not a claim about empirical frequency. It is a diagnostic space: each cell names a possible kind of meaning failure and suggests a different remediation path.

```
                Underdef Contra Frag Stale Opaque Coupled Inadm Overclaim Lossy
Lexical x x x x x - - - x
Ontological x x x x x x x x x
Boundary x x x x x x x x x
Procedural x x x x x x x x x
Evidentiary x x x x x x x x x
Authority x x x x x x x x x
Claim x x x x x x x x x
```

The sparse cells are intentional. Some combinations are less natural than others. For example, lexical inadmissibility is usually mediated through evidentiary or claim debt rather than appearing as a primary cell. The matrix is not a taxonomy of all observed incidents. It is a map of diagnostic possibility.

6.1 High-Value Matrix Examples

```
Lexical + underdefinition
  Example: GESD appears without expansion.
  Risk: operators infer meaning from context.
  Remedy: add glossary entry and link from roadmap.

Lexical + staleness
```

```
  Example: old field name preserves previous standing.
  Risk: tools read obsolete meaning as current.
  Remedy: add deprecation table and migration mapping.

Ontological + overclaim
  Example: validation is treated as admission.
  Risk: local check becomes unauthorized status ascent.
  Remedy: enforce validation/admission distinction in standing graph.

Ontological + overcoupling
  Example: one `status` field stores validation, admission, and
  execution state.
  Risk: category collapse.
  Remedy: split status into typed fields.

Boundary + lossy translation
  Example: Python package is ported to C but refusal modes vanish.
  Risk: behavior is preserved while governance semantics are lost.
  Remedy: require migration contract with refusal mapping.

Boundary + opacity
  Example: adapter boundary hides policy decision.
  Risk: authority crossing cannot be inspected.
  Remedy: extract policy surface and effect request record.

Procedural + contradiction
  Example: README says preflight while decision record grants standing.
  Risk: canonical transition cannot be identified.
  Remedy: source hierarchy and consistency gate.

Procedural + staleness
  Example: roadmap next rung points to current rung.
  Risk: process loops or false completion.
  Remedy: roadmap/index cross-check.

Evidentiary + inadmissibility
  Example: receipt lacks prior-state hash.
  Risk: claim cannot be supported.
  Remedy: require receipt schema check.

Evidentiary + fragmentation
  Example: proof split across logs, screenshots, and filenames.
  Risk: no claim-supporting evidence chain.
  Remedy: bind artifacts into receipt or manifest.

Authority + opacity
```

```
  Example: filesystem write authority inferred from environment
  variable.
  Risk: hidden permission path.
  Remedy: require explicit authority reference.

Authority + overcoupling
  Example: validation function also mutates state.
  Risk: checking becomes acting.
  Remedy: separate validator from executor.

Claim + overclaim
  Example: fixture success is described as self-hosting.
  Risk: maturity inflation.
  Remedy: active non-claim and standing gate.

Claim + contradiction
  Example: NONCLAIMS.md blocks a claim asserted by README.
  Risk: incompatible claim surfaces.
  Remedy: claim ledger evaluation.
```

A semantic debt review should classify each issue by matrix coordinate and assign a repayment action.

7. From Matrix to Governing Artifacts

The matrix becomes useful when each debt coordinate points to an artifact capable of preventing, detecting, or repaying it. These artifacts are not a second introduction to the theory. They are the enforcement layer implied by the matrix.

7.1 Glossary

Primary coverage:

```
lexical debt
underdefinition
fragmentation
staleness
lossy translation
```

The glossary defines terms, acronyms, standing states, refusal codes, policy names, claim names, and deprecated names. It prevents name drift and gives tools a canonical vocabulary.

7.2 Non-Claims File

Primary coverage:

```
claim debt
overclaim
contradiction
staleness
standing confusion
```

The non-claims file records what the system does not currently assert. It blocks unsupported maturity claims and prevents evidence from being inflated into standing.

7.3 Claim Ledger

Primary coverage:

```
claim debt
evidentiary debt
inadmissibility
contradiction
overclaim
```

The claim ledger maps claims to supporting receipts and blocking non-claims. It answers what the system may currently assert.

7.4 Standing Graph

Primary coverage:

```
ontological debt
procedural debt
claim debt
overclaim
staleness
```

The standing graph defines permissible standing transitions, revocations, downgrades, and scoped maturity claims. It prevents validation, admission, execution, and standing from collapsing into one status.

7.5 Migration Contract

Primary coverage:

```
boundary debt
procedural debt
lossy translation
inadmissibility
```

The migration contract maps meanings across versions, schemas, languages, substrates, packages, and governance levels. It prevents behavior-preserving translations from becoming meaning-losing translations.

7.6 Receipt Schema

Primary coverage:

```
evidentiary debt
authority debt
inadmissibility
opacity
fragmentation
```

The receipt schema defines what fields must be bound for a trace to become evidence. It binds state, policy, authority, artifacts, effects, refusals, and claims.

7.7 Semantic Debt Ledger

Primary coverage:

```
all subdomains
all modes
```

The semantic debt ledger records debt items by coordinate, risk, affected artifact, and repayment action. It turns the matrix into trackable engineering work.

8. Semantic Debt Ledger

A semantic debt ledger records known semantic debt items.

A minimal ledger entry should include:

```
debt_id
title
subdomain
mode
affected_artifacts
description
risk
blocked_claims
related_nonclaims
severity
repayment_action
owner
status
introduced_by
detected_by
resolved_by
```

Example:

```
debt_id: SD-LEX-001
title: GESD acronym undefined
```

```
subdomain: lexical
mode: underdefinition
affected_artifacts: roadmap, verifier report
risk: operator confusion; false standing inference
repayment_action: add GESD to GLOSSARY.md and link from roadmap
status: open
```

Example:

```
debt_id: SD-CLAIM-004
title: Receipt existence implies schema standing
subdomain: claim, ontological
mode: overclaim, category collapse
affected_artifacts: README
risk: standing inflation
blocked_claims: schema_conformant_receipt_standing
repayment_action: add non-claim and require standing transition record
status: open
```

The ledger turns semantic debt into governable work.

9. Semantic Debt Interest

Semantic debt interest is the increasing cost of future interpretation, change, migration, audit, or governance caused by unresolved semantic debt.

Forms of interest include:

- longer onboarding time;
- agent confusion;
- incorrect refactors;
- false claims;
- unsafe automation;
- migration loss;
- duplicated terminology;
- evidence reconstruction cost;
- policy drift;
- audit failure;
- standing inflation;
- inability to admit or refuse artifacts mechanically.

Semantic debt interest compounds when multiple debts interact.

Examples:

```
lexical debt + claim debt
  -> ambiguous claims

boundary debt + authority debt
  -> hidden authority leakage
```

```
evidentiary debt + procedural debt
  -> unprovable transition

ontological debt + overclaim
  -> false maturity ascent
```

The most expensive semantic debts are those that make future evidence impossible.

10. Proper and Improper Semantic Defragmentation

Semantic defragmentation is the act of reorganizing a system so its meanings become more locally recoverable, better bounded, and more admissible.

Proper semantic defragmentation:

- moves terms into a glossary;
- separates validation, admission, execution, and standing;
- extracts policy from implementation;
- binds traces into receipts;
- restores non-claims;
- names refusal modes;
- adds standing records;
- creates migration mappings;
- updates stale indexes;
- separates effects from core logic.

Improper semantic defragmentation is a refactor, cleanup, compression, abstraction, or migration that reduces apparent complexity while increasing semantic debt.

Examples:

- merging policy and execution because it is shorter;
- deleting refusal tests because success tests pass;
- replacing receipts with logs;
- removing non-claims because they seem negative;
- collapsing multiple statuses into one field;
- renaming terms without migration table;
- moving authority checks into UI code;
- treating a fixture-specific result as general standing.

Improper semantic defragmentation explains why some clean systems become less governable. They are cleaner in implementation shape and poorer in semantic shape.

11. Detection Methods

Semantic debt can be detected by human review, static tooling, dynamic checks, agent audits, and governance gates.

11.1 Lexical Detection

Checks:

- glossary coverage;
- undefined acronyms;
- deprecated term usage;
- synonym collisions;
- overloaded field names;
- term-definition drift.

11.2 Ontological Detection

Checks:

- status enum review;
- type hierarchy review;
- category-collapse detection;
- standing graph validation;
- "this is not that" invariant checks.

11.3 Boundary Detection

Checks:

- context-map review;
- adapter/core separation;
- forbidden dependencies;
- cross-substrate claim checks;
- scope field validation.

11.4 Procedural Detection

Checks:

- decision record coverage;
- roadmap/index consistency;
- standing transition records;
- migration receipt coverage;
- repair record distinction.

11.5 Evidentiary Detection

Checks:

- receipt schema conformance;
- artifact hash coverage;
- evidence chain completeness;
- trace-to-receipt distinction;
- claim-to-evidence satisfiability.

11.6 Authority Detection

Checks:

- effect-without-authority detection;
- capability surface review;
- policy totality;
- adapter mediation;
- hidden subprocess/network/filesystem access in core.

11.7 Claim Detection

Checks:

- claim ledger evaluation;
- non-claim preservation;
- unsupported claim detection;
- maturity-level separation;
- scope mismatch detection.

12. Semantic Debt and Agentic Software Development

Agentic development makes semantic debt easier to create and easier to detect.

AI agents can generate code that passes local tests while violating project meanings. They may rename concepts, merge statuses, delete comments, infer missing authority, flatten domain distinctions, or treat natural-language instructions as optional style. The result can be high functional utility with low semantic integrity.

Semantic debt theory gives agentic development a diagnostic target. It says not only "the agent violated instructions," but which semantic subdomain was damaged and by what mode.

12.1 Normative Failure Modes for Agents

Agentic systems should treat the following as first-class semantic hazards.

Acronym Drift An agent introduces or reuses an acronym without a glossary entry.

Debt coordinate:

```
lexical + underdefinition
lexical + fragmentation
```

Required guard:

```
No new acronym, standing term, refusal code, claim kind, or verifier
lane name without glossary update.
```

Status Collapse An agent collapses validation, admission, execution, and standing into a single status.

Debt coordinate:

```
ontological + overcoupling
claim + overclaim
```

Required guard:

```
Typed status fields and standing graph validation.
```

Authority Inference An agent infers permission from code reachability, filesystem access, API credentials, or prior success.

Debt coordinate:

```
authority + opacity
authority + overclaim
```

Required guard:

```
No effect admission without explicit authority reference and policy
permit.
```

Claim Inflation An agent describes a local result as a broader maturity claim.

Debt coordinate:

```
claim + overclaim
ontological + category collapse
```

Required guard:

```
Claim ledger check and active non-claim preservation.
```

Boundary Leakage An agent moves policy into adapters, imports substrate-specific behavior into core logic, or lets UI language imply governance standing.

Debt coordinate:

```
boundary + overcoupling
authority + opacity
```

Required guard:

```
Adapter isolation, context boundary checks, and migration contracts.
```

Evidence Substitution An agent treats logs, traces, summaries, or console output as evidence.

Debt coordinate:

```
evidentiary + inadmissibility
evidentiary + fragmentation
```

Required guard:

```
Only receipt-bound artifacts can support claims.
```

12.2 Agent Operating Rule

A coding agent operating under semantic debt governance should obey:

```
Do not introduce, rename, delete, merge, or promote a semantic
object unless the corresponding governing artifact is updated.
```

Examples:

```
new term -> glossary update
new claim -> claim ledger update
new limitation -> non-claim update
new status -> standing graph update
new evidence -> receipt schema or manifest update
new migration -> migration contract update
new debt -> semantic debt ledger update
```

Agentic development makes semantic debt easier to create and easier to detect. The difference is whether the project gives agents canonical surfaces to respect.

13. Relationship to Constitutional Semantic Engineering

Semantic Debt and Constitutional Semantic Engineering are paired concepts.

```
Semantic Debt
  diagnoses meaning failure

Constitutional Semantic Engineering
  builds systems to prevent, detect, and repay meaning failure
```

CSE repays semantic debt by making meaning, authority, effects, evidence, refusal, standing, and non-claims first-class artifacts.

Semantic debt explains why CSE matters. CSE provides engineering mechanisms to keep semantic debt from compounding silently.

14. Minimal Governance Model

A semantic debt governance model should include seven artifacts:

```
glossary
non-claims file
claim ledger
standing graph
migration contract
receipt schema
semantic debt ledger
```

Together these artifacts provide a minimal discipline for semantic governance.

15. Formal Sketch

Let a semantic debt item be:

$$d = \langle A, M, X, r, c \rangle$$

Where:

- A is the affected semantic subdomain.
- M is the debt mode.
- X is the affected artifact set.
- r is the risk or consequence.
- c is the corrective obligation.

Let the set of semantic subdomains be:

$$A \in \{Lex, Ont, Bound, Proc, Evid, Auth, Claim\}$$

Let the set of modes be:

$$M \in \{Underdef, Contra, Frag, Stale, Opaque, Coupled, Inadm, \\ Overclaim, Lossy\}$$

A semantic debt ledger is then:

$$D = \{d_1, d_2, ..., d_n\}$$

A system is semantically admissible for a claim k only if:

$$supports(k) \land \neg blocked(k) \land \neg critical_debt(k, D)$$

Where $critical_debt(k, D)$ holds when unresolved debt items affect the semantic surfaces needed to support k.

For example, a claim of schema-conformant receipt standing is blocked if unresolved debt affects receipt schema definition, receipt binding, standing transition records, or active non-claims.

Semantic debt is therefore not only descriptive. It can participate in claim evaluation.

16. Open Problems

Several problems remain open.

16.1 Severity Metrics

How should semantic debt severity be measured? Possible factors include claim impact, authority impact, migration impact, evidence impact, agent confusion risk, and cost to repair.

16.2 Automated Debt Detection

Which forms of semantic debt can be detected statically? Lexical underdefinition and stale references may be straightforward. Ontological collapse and claim inflation may require stronger semantic models.

16.3 Semantic Debt Interest

How should interest be estimated? Semantic debt may compound nonlinearly when lexical, ontological, evidentiary, and claim debts interact.

16.4 Cross-System Debt Propagation

How does semantic debt propagate through APIs, packages, schemas, model outputs, documentation, and agent instructions?

16.5 Non-Claim Synthesis

Can systems automatically infer appropriate non-claims from missing evidence, failed tests, incomplete migrations, or unsupported standings?

16.6 Semantic Debt in Machine Learning Systems

ML systems introduce additional semantic debt through dataset labels, prompt instructions, embeddings, latent representations, evaluation gaps, and benchmark overclaims.

16.7 Formal Ontology Standard

The proposed ontology may be developed into a machine-readable standard for debt ledgers, claim ledgers, and governance checks.

17. Conclusion

Semantic debt is the debt of meaning that can no longer be safely recovered.

It appears when terms drift, categories collapse, boundaries leak, procedures go unrecorded, evidence becomes inadmissible, authority hides, and claims outrun support. It is not merely poor documentation, nor merely technical debt under another name. It is the accumulated future cost of unstable meaning.

The ontology proposed here separates semantic debt into seven subdomains: lexical, ontological, boundary, procedural, evidentiary, authority, and claim debt. It also identifies nine cross-cutting modes: underdefinition, contradiction, fragmentation, staleness, opacity, overcoupling, inadmissibility, overclaim, and lossy translation. This two-axis model clarifies why semantic fragmentation is not a peer subdomain but an orthogonal distribution failure. The more precise subtype originally motivating the framework is semantic locality debt: the loss of stable homes for meaningful distinctions.

The matrix is the paper's central instrument. It turns semantic debt into a coordinate system. The governing artifacts then turn that coordinate system into practice: glossary, non-claims file, claim ledger, standing graph, migration contract, receipt schema, and semantic debt ledger.

The result is a governance model. Semantic debt can be recorded, classified, prioritized, checked, repaid, and blocked from supporting claims. It can become part of admission, refactor, migration, and agent-development workflows.

Semantic debt names the cost of unrecoverable meaning. Constitutional Semantic Engineering offers one path for refusing it before it compounds.

Corpus-Wide Reference Positioning

This paper argues that software systems do not only accumulate technical debt. Its nearest external comparison fields are software, semantic, governance. The sources in [E1], [E2], [E3], [E4], [E5], [E6], [E7] are cited as predecessor literature, adjacent standards, or counterpressure - not as evidence that they caused the Ableman framework or that they validate its novel claims. The comparison isolates where established work already supplies provenance, formal assurance, institutional constraint, empirical method, or engineering practice, and where this paper adds a distinct claim about admissibility, authority, semantic custody, or governed realization.

Within the Ableman corpus, the highest-priority current cross-links are [A1], [A2], [A3], [A4], [A5], [A6], [A7]. These are links among separately custodied

works, not silent identity mergers. Their role is to expose mutual dependencies, terminology collisions, and places where a reader should traverse the corpus before treating this paper as self-sufficient.

Reference-pass counterevidence and limit search

The pass applied the following pressure tests: Modularity, secure-development practice, and reproducibility reduce classes of failure but do not guarantee architectural coherence or user sovereignty. Formal vocabularies and graph links make commitments explicit but cannot by themselves establish correct identity, shared interpretation, or lossless translation between contexts. A written constitution can legitimate domination as easily as constrain it unless amendment, participation, enforcement, and appeal remain effective in practice. These limits are not generic disclaimers: they identify the assumptions that a later empirical, formal, or implementation review must try to break.

Reference-pass influence and novelty boundary

No newly added external source is asserted to be a documented authorial influence unless the predecessor source already said so. `Predecessor` means earlier relevant work;

`adjacent` means a useful comparison; `standard` means an interoperability or assurance baseline; and `counterevidence` marks work that narrows or pressures a claim. Novelty therefore remains a reviewable, claim-level question rather than an inference from a low similarity score or an absent citation.

References

External scholarship, standards, and prior art

[**E1**] David L. Parnas. "On the Criteria To Be Used in Decomposing Systems into Modules." *Communications of the ACM* 15(12), 1972, pp. 1053-1058. DOI: 10.1145/361598.361623. https://doi.org/10.1145/361598.361623 **Relationship:** Foundational modularity treatment used to compare information hiding with explicit semantic and governance interfaces.

[**E2**] Ward Cunningham. "The WyCash Portfolio Management System." *OOPSLA '92 Experience Report*, 1992. DOI: 10.1145/157709.157715. https://doi.org/10.1145/157709.157715 **Relationship:** Original technical-debt metaphor used to distinguish implementation debt from semantic debt.

[**E3**] Karen Scarfone, Murugiah Souppaya, and Donna Dodson. *Secure Software Development Framework (SSDF) Version 1.1.* NIST SP 800-218, 2022. DOI: 10.6028/NIST.SP.800-218. https://doi.org/10.6028/NIST.SP.800-218 **Relationship:** Adjacent secure-development framework for preparation, protection, production, and vulnerability response.

[**E4**] Reproducible Builds Project. *Formal Definition of Reproducible Builds*. reproducible-builds.org source **Relationship:** Adjacent reproducibility practice; byte reproduction is compared with, but not equated to, semantic or constitutional continuity.

[**E5**] Richard Cyganiak, David Wood, and Markus Lanthaler, eds. *RDF 1.1 Concepts and Abstract Syntax*. W3C Recommendation, 25 February 2014. https://www.w3.org/TR/rdf11-concepts/ **Relationship:** Adjacent graph data model and identity baseline.

[**E6**] Alistair Miles and Sean Bechhofer, eds. *SKOS Simple Knowledge Organization System Reference*. W3C Recommendation, 18 August 2009. https://www.w3.org/TR/skos-reference **Relationship:** Adjacent standard for concepts, labels, schemes, notes, and semantic mappings.

[**E7**] W3C OWL Working Group. *OWL 2 Web Ontology Language Document Overview*, Second Edition. W3C Recommendation, 11 December 2012. https://www.w3.org/TR/owl2-overview/ **Relationship:** Adjacent formal ontology standard used to compare declared semantics and inference commitments.

Ableman corpus

[**A1**] Adam Ableman. *Sovereign Software*. Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-24C6159B98023A9C`;
source SHA-256
`6AAFE79E438B980B75D1910A897F7BA716A10CF1187B68CB1F71405F36725A18`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A2**] Adam Ableman. *Constitutional Software Analysis (CSoftA): A Governance-Based Classification Method for Software Systems*. Ableman Corpus current edition; work identity `PAPER.PUB.20472196`;
source SHA-256
`3670B28B7C1AF5467BE4002C47B5F964F8FAF6F257125C2F84CE06904E4FD0DE`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A3**] Adam Ableman. *Semantic Fragmentation and Constitutional Structure in Software Systems*. Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-B307F65B51C503DE`;
source SHA-256
`9D2A9D79DE8BC3F2F8ABECA073E4D3607F7623F4368E63F8036081E39B4C75C3`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A4**] Adam Ableman. *Governed Coding and Debugging*. Ableman Corpus current edition; work identity `PAPER.PUB.18115533`;
source SHA-256

2469EDA96DBC1E207855A9AD27314A7FAB59FA35E6EFEF9288C30F949376BF16.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A5**] Adam Ableman. *Constitutional Operating Systems.* Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-F5FB8FA82270B989;
source SHA-256
E982E4D6BD4B519EED6A7438367844954D6D5F1C2C4CC0099AAAC4AB9A00D3AB.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A6**] Adam Ableman. *Meaning as Binding.* Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-5F976917F0B283EA;
source SHA-256
A470D1A0B9ED7C3CEAB6BB88BE4B642A2BA36427447F787764329FD574DA0C97.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A7**] Adam Ableman. *Governed Meaning Base (GMB).* Ableman Corpus current edition; work identity
AMCS-LOCAL-WORK-D95CCC4FB7B58BFB;
source SHA-256
A3B62E50F8BEE125FDAE0D38C2839AFD1CCC138296268BF93217481E38AA48EA.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

Editorial source notes retained from predecessor

Avgeriou, P., Ozkaya, I., Chatzigeorgiou, A., Ciolkowski, M., Ernst, N. A., Koontz, R. J., Poort, E., and Shull, F. (2024). Technical Debt Management: The Road Ahead for Successful Software Delivery. arXiv. https://arxiv.org/abs/2403.06484

Cockburn, A. (2005). Hexagonal Architecture: The Original 2005 Article. HaT Technical Report 2005.02. https://alistair.cockburn.us/hexagonal-architecture

Dorval, G. (2026). Semantic Debt: Canonical Definition. gautierdorval.com source

Evans, E. (2003). Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley Professional.

Jansen, A. G. J. (2008). Architectural Design Decisions. University of Groningen doctoral thesis. research.rug.nl source

Kruchten, P., Nord, R. L., and Ozkaya, I. (2012). Technical Debt: From Metaphor to Theory and Practice. IEEE Software, 29(6), 18-21. DOI: 10.1109/MS.2012.167.

Lebo, T., Sahoo, S., McGuinness, D., Belhajjame, K., Cheney, J., Corsar, D., Garijo, D., Soiland-Reyes, S., Zednik, S., and Zhao, J. (2013). PROV-O: The PROV Ontology. W3C Recommendation. https://www.w3.org/TR/prov-o/

Miller, M. S., Yee, K. P., and Shapiro, J. S. (2003). Capability Myths Demolished. Systems Research Laboratory, Johns Hopkins University. http://srl.cs.jhu.edu/pubs/SRL2003-02.pdf

Nogueira, F., Silva, N., and Conte, T. (2026). One Size Fits All? An Empirical Comparison of ADR Templates regarding Comprehension, Usability, and Ease of Adoption. arXiv. https://arxiv.org/abs/2604.27333

Ozkan, O., Babur, O., and van den Brand, M. (2023). Domain-Driven Design in Software Development: A Systematic Literature Review on Implementation, Challenges, and Effectiveness. arXiv. https://arxiv.org/abs/2310.01905

Parnas, D. L. (1972). On the Criteria to Be Used in Decomposing Systems into Modules. Communications of the ACM, 15(12), 1053-1058. DOI: 10.1145/361598.361623.

Rios, N., Ribeiro, L. F., Caires, V., Mendes, T., and Spindola, R. O. (2014). Towards an Ontology of Terms on Technical Debt. IEEE International Workshop on Managing Technical Debt. DOI: 10.1109/MTD.2014.9.

Sharma, R. K. (2026). ContextCov: Deriving and Enforcing Executable Constraints from Agent Instruction Files. arXiv. https://arxiv.org/abs/2603.00822

Tofan, D. (2015). Understanding and Supporting Software Architectural Decisions: For Reducing Architectural Knowledge Vaporization. University of Groningen doctoral thesis. research.rug.nl source

---

Corpus source and companion formats: https://github.com/AblemanMosaic/ableman-corpus
License: CC-BY-ND-4.0

DOI: 10.5281/zenodo.21111666